

TinyML on ESP32-S3: Machine Learning Architecture Report

1. Purpose and Scope

This section defines the machine-learning subsystem for the Hybrid Mesh Micro-Climate Wireless Sensor Network. The subsystem performs local anomaly detection on each ESP32-S3 slave node so that routine measurements do not require cloud connectivity or continuous radio transmission.

The model classifies environmental conditions into five operational categories: Normal, Rain, Thunderstorm, High PM2.5, and Radiation Spike. Its output controls the node’s communication behavior: normal readings are transmitted only during scheduled polling, while detected anomalies can trigger an immediate authenticated alert. [file:1]

2. TinyML Requirements

The model must satisfy the following embedded constraints:

- Execute locally on an ESP32-S3 at 240 MHz.
- Complete inference within the project’s active-cycle budget.
- Use limited RAM and flash memory.
- Support mixed environmental, weather, particulate, acoustic, and radiation features.
- Produce deterministic output suitable for firmware decision-making.
- Operate without cellular, cloud, or paid online services.
- Integrate with the node’s deep-sleep and radio-power-gating strategy.

The design document estimates XGBoost inference at approximately 15 ms and places it inside a sequential firmware pipeline whose complete active cycle is targeted at less than 200 ms. [file:1]

3. Sensor Data Sources

The model receives data derived from the node’s 11-sensor array:

Sensor or source	Example model information
BME680	Temperature, relative humidity, pressure, and air-quality-related readings
AS3935	Lightning detection and estimated distance or energy
NEO-6M GPS	Deployment position and time registration; normally not a frequent inference input
Anemometer	Wind speed
Pluviometer	Rainfall rate or bucket-tip accumulation

Sensor or source	Example model information
BPW34 + LM358	Radiation pulse/count-rate information
INMP441 + LM393	Acoustic event information such as thunder or chainsaw-like activity
PMS5003	PM1.0, PM2.5, and PM10 concentration
AS5600	Wind direction
LTR390-UV	UVA and ambient-light measurements
SHT20	Soil temperature and soil-moisture-related reading

The project document specifies sequential sensor acquisition, power gating for high-consumption peripherals, and packing of the readings into a sensor data structure before preprocessing. [file:1]

4. Input Feature Engineering

The proposed input vector contains 22 values:

$$X_t = [a_1, a_2, \dots, a_{11}, d_1, d_2, \dots, d_{11}]$$

where:

- a_i is the normalized absolute value of sensor feature i at time t .
- d_i is the normalized change from the previous measurement.

The delta is calculated as:

$$d_i = v_i(t) - v_i(t - 1)$$

The previous readings should be stored in RTC memory so they survive ESP32 deep-sleep cycles. The same preprocessing logic must be used during both training and deployment; otherwise, the firmware model will receive inputs that do not match the inputs used during training. [file:1]

Recommended preprocessing sequence

1. Wake the node and restore the previous feature vector from RTC memory.
2. Power only the required sensor groups.
3. Read the 11 sensor sources sequentially.
4. Validate readings and mark unavailable values explicitly.
5. Calculate absolute and delta features.
6. Apply fixed training-time min-max parameters.
7. Clamp values to the expected range.
8. Store the current readings in RTC memory.
9. Pass the 22-value vector to the XGBoost inference function.

For each feature, store the training-time minimum and maximum:

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$$

If $x_{max} = x_{min}$, the feature should be assigned a safe constant such as 0.0 rather than causing a division-by-zero error. Values outside the training range should be clamped to [0, 1].

5. Dataset Design

The development pipeline specifies at least 500 labeled samples covering five anomaly classes and at least two weeks of training-data collection. [file:1] This should be treated as a minimum prototype target rather than a final production dataset.

Dataset record format

Each record should contain:

- Node ID and deployment location.
- UTC timestamp.
- Raw readings from all available sensors.
- Previous-cycle readings.
- Derived delta features.
- Battery voltage and sensor validity flags.
- Environmental context, such as rain or deployment conditions.
- Human-verified class label.
- Optional event confidence and annotation notes.

A practical CSV or Parquet schema is:

```
node_id,timestamp,temp,rh,pressure,pm25,rain_rate,wind_speed,
wind_direction,radiation_cpm,lightning_energy,acoustic_score,
soil_temp,soil_moisture,uv,prev_temp,prev_pm25,...,label
```

The production training table should ultimately contain the exact 22 features used by firmware, along with the label and a record-quality field.

Required classes

Class	Example event	Important signals
Normal	Stable environmental conditions	Small feature deltas and no validated event indicator
Rain	Active rainfall	Pluviometer tips, humidity, pressure, acoustic/lightning context
Thunderstorm	Lightning or thunder event	AS3935, acoustic features, pressure, rain, wind
High PM2.5	Particulate pollution event	PMS5003 PM2.5/PM10, wind, humidity, acoustic or weather context

Class	Example event	Important signals
Radiation Spike	Radiation count-rate deviation	BPW34 count rate, filtered baseline, rain and pressure context

The dataset should include gradual transitions, noisy measurements, sensor startup artifacts, and simultaneous events. For example, rainfall and radiation-count changes should not automatically be labeled as a nuclear or industrial anomaly because the document identifies precipitation-related radon washout as a potential confounder. [file:1]

6. Collection and Labeling Strategy

Use a two-stage approach:

Stage 1: Instrumentation and baseline collection

Collect continuous raw data from all nodes without enabling autonomous radio alerts. Record normal day/night variation, temperature changes, humidity cycles, wind variation, particulate variation, and sensor noise.

Stage 2: Controlled event collection

Add labeled event windows using real observations and safe controlled demonstrations where possible. Do not create hazardous radiation, fire, or severe-weather conditions for data collection. Event labels should be assigned using external evidence such as a rain gauge, trusted weather observation, calibrated particulate reference, lightning observation, or verified acoustic recording.

Each event should include a pre-event window, event window, and recovery window. This prevents the classifier from learning only the peak value and improves detection of onset and recovery behavior.

7. Model Selection

The proposed model is XGBoost with a compact decision-tree ensemble compiled or ported into C++ firmware. The model is suitable for this application because it can represent nonlinear relationships among heterogeneous sensor values and does not require floating-point neural-network kernels or a large runtime framework.

The document's acceptance target is an F1 score of at least 0.88, with an approved confusion matrix, followed by verification that embedded outputs match a Python reference implementation on identical inputs. [file:1]

Recommended training constraints

- Use shallow trees to control flash, RAM, and inference time.
- Limit the number of estimators after measuring embedded performance.
- Use class weighting or balanced sampling for rare anomaly classes.
- Optimize for macro-F1 and recall of safety-relevant anomalies, not accuracy alone.

- Keep a separate validation set grouped by event and time period.
- Export the final model together with preprocessing constants and class mapping.

A small decision-tree model is preferable to a larger model with slightly better offline accuracy if the larger model increases false alerts, active time, or firmware complexity.

8. Train–Deploy Pipeline

```

Sensor firmware
  |
  v
Raw synchronized records
  |
  v
Quality checks and event labeling
  |
  v
Absolute + delta feature generation
  |
  v
Training-only split by event/time/node
  |
  v
XGBoost training and hyperparameter tuning
  |
  v
Validation: macro-F1, recall, confusion matrix
  |
  v
Model export + min/max constants + class map
  |
  v
C/C++ embedded inference implementation
  |
  v
Python-versus-firmware equivalence testing
  |
  v
ESP32-S3 field validation

```

The preprocessing implementation should be treated as part of the model artifact. Deploying only the tree model without its scaling constants, feature order, missing-value policy, and class mapping is unsafe because the firmware may produce valid-looking but incorrect predictions.

9. Embedded Inference Interface

A suitable firmware interface is:

```

enum AnomalyClass : uint8_t {
    NORMAL = 0,
    RAIN = 1,
    THUNDERSTORM = 2,
    HIGH_PM25 = 3,

```

```
RADIATION_SPIKE = 4
};

struct InferenceResult {
    uint8_t anomaly_class;
    uint8_t confidence;
    uint8_t valid_features;
};

InferenceResult run_inference(const float features[22]);
```

The inference function should not directly transmit packets or operate the radio. It should return a result to the sequential task, which applies the AI gate, records diagnostics, and then decides whether to transmit, wait for polling, or enter deep sleep.

10. Handling Missing and Unreliable Inputs

Sensor failures must not silently become ordinary zero values. Each feature should have a validity flag, and the model should be trained with representative missing or degraded-input cases if the deployed hardware can experience them.

Recommended policies include:

- Reject impossible readings before scaling.
- Use the last valid value only for short, explicitly bounded outages.
- Add sensor-validity indicators if missing data is common.
- Disable or isolate a broken peripheral through the existing RPC power-gating mechanism.
- Log the feature-validity mask with every anomaly decision.

The model should also distinguish “sensor unavailable” from “measured zero.” This is especially important for radiation, rainfall, acoustic, and particulate channels.

11. Radiation and Acoustic Submodels

Radiation detection should be treated as environmental anomaly detection rather than certified dosimetry. The document identifies the BPW34 detector as an uncalibrated relative counter and recommends calibration against a known reference before making quantitative claims. [file:1]

For radiation features, calculate a filtered baseline and deviation from baseline. Include rainfall and barometric pressure because precipitation can produce natural radiation-count changes through radon washout. For acoustic classification, a lightweight approach is to use a short FFT or engineered spectral features and pass an acoustic score or event flag into the main classifier rather than running a large audio neural network continuously.

12. Evaluation Plan

Evaluate the model at three levels:

Offline Python evaluation

Measure macro-F1, per-class precision, recall, confusion matrix, false-alert rate, and missed-alert rate. Split by event rather than randomly splitting adjacent samples, because random row splits can leak nearly identical measurements into both training and validation sets.

Firmware equivalence evaluation

Run the same serialized feature vectors through Python and the ESP32 implementation. Confirm identical class outputs and acceptable confidence differences for floating-point calculations. Test boundary values, missing values, normalization limits, and malformed model data.

Field evaluation

Run the system for at least 72 hours as specified in the project pipeline, recording raw inputs, model outputs, radio decisions, battery voltage, and sensor validity. [file:1] Review every false alert and missed event before enabling unattended alerting.

13. Risks and Mitigations

Risk	Impact	Mitigation
Too few labeled anomaly samples	Poor recall and unstable class boundaries	Extend collection, use event-based splits, collect varied conditions
Class imbalance	Model predicts Normal too often	Class weights, balanced sampling, recall-focused tuning
Sensor drift	Gradual accuracy loss	Periodic calibration and baseline monitoring
Rain/radiation correlation	False radiation alerts	Add filtered baseline, rain rate, pressure, and conservative thresholds
Feature-order mismatch	Incorrect inference	Version feature schema and test serialized vectors
Model too large	Excess flash/RAM or latency	Limit tree depth and estimators; benchmark on hardware
Startup artifacts	False PM or audio alerts	Respect warm-up times and add validity flags
Cold-start operation	No trained model initially	Use rule-based thresholds until labeled data is sufficient

14. Final Recommendation

Proceed with XGBoost as the primary TinyML model for the prototype, but define the deployed artifact as a bundle containing the model, feature order, scaling constants, class mapping, missing-value policy, and version number. The 22-feature absolute-plus-delta representation is appropriate for the low-power architecture, provided that RTC state persistence and sensor validity handling are implemented correctly.

Before field deployment, prioritize three experiments: establish a clean labeled dataset, measure the exact model footprint and latency on the ESP32-S3, and compare Python predictions against firmware predictions on a fixed test corpus. The project should keep a conservative rule-based fallback enabled until the model meets the required per-class recall and survives the 72-hour field validation. [file:1]